SIMATS ENGINEERING

Short Answer Test

COURSE NAME: Software Testing **COURSE CODE: CSA37**

COURSE OUTCOME COVERED

CO1: *Apply software testing principles to enhance the effectiveness of software quality assurance and improve defect detection strategies.*

Assessment Tool Weightage: 30%

Sl. No	Questions	Bloom's Level
1	A startup team relies on random testing methods, leading to inconsistent results and missed defects. The project manager wants to improve testing effectiveness. As a test engineer, apply the concept of testing as an engineering activity to redesign the approach. Explain how structured test design improves quality and defect detection.	3
2	A banking system undergoes multiple failures due to lack of a defined testing process. The organization plans to standardize its testing activities. Illustrate how a well-defined testing process improves software quality in such real-time systems. Explain the benefits with relevant examples.	3
3	During development of an e-commerce platform, testing activities are performed inconsistently across phases. This leads to integration and system-level defects. Demonstrate how testing should be carried out as a structured process in SDLC. Explain how process discipline ensures better quality outcomes.	3
4	A login system shows mismatch between expected and actual outputs during validation. The development team is unsure whether the issue is an error, defect, or failure. Apply basic testing definitions to classify the issue. Explain the relationship between these terms with suitable examples.	3
5	A team struggles to detect defects early, leading to increased cost during later stages. The manager wants to improve defect detection efficiency. Apply a software testing principle to address this issue. Explain how the principle improves testing effectiveness with an example.	3
6	During requirement analysis of a healthcare system, several ambiguities go unnoticed. Later, these lead to defects in implementation. Show how a tester can contribute during the requirement phase to improve quality. Explain the importance of early testing involvement.	3

7	A payroll system produces incorrect outputs due to unclear and ambiguous requirements. The development team needs to identify the origin of defects. Analyze how such defects originate in the software lifecycle. Explain how requirement clarity can prevent such issues.	3

8	A system crashes when boundary input values are entered by users. The team suspects improper testing coverage. Classify this defect and justify your answer. Explain how boundary value analysis could have prevented this issue.	3
9	A company maintains a defect repository but rarely uses it for improving testing practices. This results in repeated defects across releases. Demonstrate how a defect repository can support better test case design. Explain its role in improving software quality.	3
10	An ATM system fails during withdrawal due to incorrect handling of transaction logic. The issue was not detected during testing. Apply your testing knowledge to identify and describe this as a functional defect. Explain how it could have been prevented.	3
11	A large application shows that most defects are concentrated in a few modules. The testing team wants to optimize their efforts. Use the concept of defect clustering to identify testing priorities. Explain how this improves testing efficiency.	3
12	A web application experiences slow performance under heavy user load. The issue needs to be categorized properly. Apply defect classification techniques to identify this type of defect. Explain how classification helps in better resolution.	3
13	A team is preparing for regression testing after multiple releases. They have access to historical defect data. Show how this data can be reused effectively for regression testing. Explain how it improves test coverage and efficiency.	3
14	During integration testing, multiple defects are found due to poor communication between developers and testers. The organization wants to improve collaboration. Identify the role of a tester in minimizing such defects. Explain how early involvement helps.	3
15	A software product repeatedly faces similar defects in every release cycle. The team needs a long-term solution. Apply defect prevention techniques to address recurring issues. Explain how prevention improves overall software quality.	3
16	A mobile application frequently crashes due to inconsistent testing approaches. The testing team wants to standardize practices. Apply testing principles to redesign the testing strategy. Explain how structured approaches improve reliability.	3

17	A company confuses debugging with testing, leading to delays in defect resolution. The team needs clarity on both concepts. Differentiate testing and debugging using a real-time scenario. Explain their roles in improving software quality.	3
18	An online shopping system behaves differently across browsers. This leads to customer dissatisfaction. Apply testing principles to design a cross-browser testing strategy. Explain how such issues can be effectively handled.	3

19	A logistics application produces inconsistent outputs due to improper validation. The issue is not detected early. Apply testing concepts to identify gaps in validation. Explain how proper testing improves system reliability.	3
20	A team skips structured test case design and directly executes random tests. This leads to poor defect detection. Apply testing as an engineering activity to improve the process. Explain the importance of systematic test design.	3
21	A healthcare system shows data inconsistency between modules due to integration issues. The testing team needs to analyze the problem. Apply testing principles to identify root causes. Explain how proper testing ensures data consistency.	3
22	A project experiences delays because testers are not involved early in development. The organization wants to improve collaboration. Explain the role of testers across SDLC phases. Show how early involvement improves quality.	3
23	A system fails during peak usage due to untested scenarios. The testing team missed critical cases. Apply testing principles to identify gaps. Explain how better test coverage prevents such failures.	3
24	A software team lacks proper defect documentation, making tracking difficult. The organization wants to improve defect management. Apply defect repository concepts to solve this issue. Explain how documentation enhances tracking and analysis.	3
25	A banking system produces repeated transaction failures due to improper testing practices. The team needs to improve quality. Apply software testing principles to redesign the testing approach. Explain how structured testing enhances reliability.	3
26	A system shows failures due to missing edge case testing. The testers focused only on normal inputs. Apply testing principles to include edge cases. Explain how boundary testing improves defect detection.	3

27	A development team struggles to maintain consistency in testing across modules. This leads to uneven quality. Apply process-based testing approaches to address this issue. Explain how standardization improves outcomes.	3
28	A web application fails due to improper handling of unexpected inputs. The testing team missed validation scenarios. Apply testing concepts to improve input validation. Explain how proper testing ensures robustness.	3
29	A software company faces frequent defect leakage into production. The management wants to reduce such issues. Apply testing principles to analyze the root cause. Explain how improved testing practices prevent leakage.	3
30	A project team is unable to identify defects early due to poor testing strategies. This increases overall cost and effort. Apply testing principles to redesign the strategy. Explain how early defect detection improves efficiency.	3

1. Testing as an Engineering Activity

Testing should be treated as a systematic engineering activity rather than random execution of test cases. The startup team should first understand the requirements, identify important functions, design test cases using suitable techniques, execute them, record defects, and evaluate the results. Structured test design helps testers cover normal, boundary, invalid, and exceptional conditions systematically. This improves defect detection because testing is based on defined objectives and coverage rather than chance. It also makes the testing process repeatable and measurable, resulting in better software quality and reliability.

2. Well-Defined Testing Process

A well-defined testing process provides a systematic approach for testing a banking system and reduces the possibility of important defects being missed. The process can include requirement analysis, test planning, test case design, test execution, defect reporting, retesting, and regression testing. For example, in an ATM system, withdrawal, balance enquiry, PIN validation, and transaction cancellation can be tested systematically. A standardized process ensures that important banking functions are tested consistently and helps identify defects before they affect customers, thereby improving software quality and reliability.

3. Testing as a Structured Process in SDLC

Testing should be performed throughout the Software Development Life Cycle rather than only at the end of development. During the requirement phase, testers review requirements for completeness and ambiguity. During design and coding, appropriate reviews and unit testing are performed, followed by integration testing when modules are combined. System testing validates the complete e-commerce application, while acceptance testing checks whether it satisfies user requirements. Following this structured process helps identify defects at the appropriate stage and prevents integration and system-level problems from accumulating until the final stage.

4. Error, Defect and Failure

An error is a human mistake made while developing or designing software, such as a programmer writing an incorrect condition. When this mistake exists in the software and causes the software to behave incorrectly, it is called a defect. When the defect is executed and the actual output differs from the expected output, it results in a failure. For example, if a developer incorrectly writes a login condition, the incorrect code represents a defect. When a valid user is unable to log in because of that defect, the incorrect behavior is a failure. Thus, an error can introduce a defect, and the execution of a defect can produce a failure.

5. Early Testing

The principle of early testing can be applied to detect defects as early as possible in the development process. Testers should review requirements and design documents before implementation begins and should start preparing test cases early. For example, if an e-commerce requirement incorrectly states how discounts should be calculated, identifying the problem during requirement review is much cheaper than discovering it after the application has been developed and deployed. Early defect detection reduces rework, saves time and cost, and improves the overall effectiveness of testing.

6. Tester Involvement During Requirements

A tester should participate during the requirement analysis phase to identify ambiguity, missing information, inconsistencies, and unrealistic requirements. In a healthcare system, requirements related to patient records, medication information, and data validation must be clearly defined because mistakes can affect later implementation. The tester can review the requirements and ask questions about unclear conditions before development begins. Early testing involvement helps prevent defects from being introduced into the system and reduces the cost and effort required to correct them later.

7. Defects from Ambiguous Requirements

Defects can originate at any stage of the software lifecycle, and unclear requirements are one of the major sources. If a payroll requirement does not clearly specify how overtime, deductions, or taxes should be calculated, developers may interpret it differently and implement incorrect logic. The resulting incorrect output becomes a defect in the software. Clear, complete, consistent, and testable requirements help prevent such problems. Requirement reviews involving developers, testers, and customers can identify ambiguities before implementation and therefore reduce defects.

8. Boundary Value Analysis

A system that crashes when boundary input values are entered may contain a boundary-related defect. This can happen when testers test only normal values and ignore minimum, maximum, just-below, and just-above boundary values. Boundary Value Analysis specifically focuses on these conditions. For example, if an age field accepts values from 18 to 60, testing should include 17, 18, 19, 59, 60, and 61. Testing these values could reveal incorrect validation or system crashes and help prevent boundary-related failures.

9. Defect Repository

A defect repository stores information about defects identified during testing, such as the defect description, affected module, severity, cause, status, and resolution. The team can analyze previous defects to identify frequently failing areas and design stronger test cases for them. For example, if previous releases repeatedly contained payment-related defects, testers can create additional test cases for payment processing during future releases. Using historical defect information improves test case design, supports regression testing, and reduces the possibility of similar defects being repeated.

10. Functional Defect in ATM

An ATM withdrawal failure caused by incorrect transaction logic can be classified as a functional defect because the system is not performing a required function correctly. For example, if the account has sufficient balance but the ATM incorrectly rejects the withdrawal, the withdrawal function does not satisfy its expected behavior. This defect could have been prevented through proper functional testing using valid withdrawals, insufficient balance, invalid amounts, daily limits, and transaction cancellation scenarios. Requirement-based test cases and regression testing would also help identify such problems before

deployment.

11. Defect Clustering

Defect clustering means that a small number of modules often contain a large proportion of the defects. If testing shows that certain modules of a large application contain many defects, those modules should receive greater testing attention. For example, if the payment and authentication modules contain most of the reported defects, testers should prioritize detailed testing and regression testing in these areas. This approach helps optimize testing resources because more effort is focused on areas with a higher probability of failure.

12. Performance Defect

A web application that becomes slow under heavy user load can be classified as a performance defect. The system may work correctly with a small number of users but fail to provide acceptable response time when many users access it simultaneously. Performance testing, load testing, and stress testing can be used to identify such problems. Proper defect classification helps the team understand the nature of the issue and assign it to the appropriate developers for resolution, improving the application's performance and reliability.

13. Historical Defect Data for Regression Testing

Historical defect data can be reused to improve regression testing after multiple releases. Previously detected defects can be converted into permanent regression test cases so that the same problems are checked whenever changes are made to the software. For example, if a previous release contained a defect in payment processing, the corresponding test case should be included in every future regression test cycle. This improves test coverage, prevents defect recurrence, and reduces the time required to identify previously experienced problems.

14. Tester Role in Integration Testing

Testers help minimize integration defects by understanding how different modules interact and by communicating potential integration risks early with developers. During integration testing, testers verify whether data is transferred correctly between modules and whether combined components behave as expected. For example, in an e-commerce system, the tester can verify whether customer information from the login module is correctly passed to the payment and order modules. Early involvement improves

communication, allows problems to be discussed before implementation is complete, and reduces integration defects.

15. Defect Prevention

Defect prevention focuses on identifying and removing the causes of defects rather than repeatedly fixing the same problems after they occur. If similar defects appear in every release, the team should analyze their root causes and improve the development or testing process. For example, repeated input-validation defects may indicate that validation requirements are unclear or that developers lack a common validation standard. Requirement reviews, coding standards, checklists, root-cause analysis, and developer training can help prevent recurring defects and improve long-term software quality.

16. Structured Testing for Mobile Applications

A mobile application with frequent crashes requires a structured and consistent testing strategy. Testers should define test conditions, prepare test cases, execute them systematically, record defects, and perform regression testing after fixes. The application should also be tested under different devices, operating systems, screen sizes, network conditions, and user actions. A structured approach ensures that important scenarios are not missed and makes testing repeatable. This improves reliability and reduces the chances of crashes reaching users.

17. Testing and Debugging

Testing and debugging are related but different activities. Testing is performed to identify failures or defects in software, while debugging is performed by developers to locate the cause of a defect and correct the code. For example, if an ATM gives an incorrect balance after a withdrawal, the tester reports the failure with the relevant test details. The developer then debugs the program to find the incorrect transaction logic and fixes it. After the correction, the tester performs testing again to verify that the defect has been resolved.

18. Cross-Browser Testing

Cross-browser testing is required when a web application behaves differently across browsers. The tester should execute important functional and UI test cases on supported browsers such as Chrome, Edge, Firefox, and other required environments. The same inputs and expected results should be compared across browsers. Differences in layout, JavaScript execution, compatibility, or performance should be recorded as defects.

Systematic cross-browser testing ensures that customers receive consistent functionality and user experience regardless of the supported browser they use.

19. Validation in Logistics Application

Inconsistent outputs in a logistics application may indicate gaps in validation and test coverage. The tester should verify whether requirements are correctly understood and whether inputs, processing rules, and outputs are properly validated. Different valid, invalid, boundary, and unexpected inputs should be tested to identify missing validation conditions. Proper testing helps detect incorrect calculations, missing checks, and inconsistent results before deployment. This improves the reliability of logistics operations and ensures that the application produces accurate results under different conditions.

20. Systematic Test Design

Random testing alone cannot provide sufficient and consistent coverage because important scenarios may be missed. Testing as an engineering activity requires systematic test design based on requirements, risks, and suitable testing techniques. The team should identify test conditions and create test cases covering valid, invalid, boundary, and exceptional situations. For example, a registration form should be tested with valid information as well as missing, incorrect, and boundary inputs. Systematic test design improves coverage, makes testing repeatable, and increases the probability of detecting important defects.

21. Integration Issues and Data Consistency

Data inconsistency between healthcare system modules can occur when integrated components use different data formats, validation rules, or communication methods. Testers should verify data flow between modules and check whether information remains accurate when transferred from one component to another. For example, patient information entered in one module should appear correctly in the billing or medical-record module. Integration testing, interface testing, and end-to-end testing can identify such problems. Proper testing ensures that data remains consistent and reliable across the complete system.

22. Tester Role Across SDLC

Testers should be involved throughout the SDLC rather than joining the project only after coding is completed. During requirements, they can identify ambiguity and missing conditions; during design, they can review testability and possible risks; during development, they can prepare test cases and support unit and integration testing; and during system testing, they validate complete functionality. Early involvement

allows defects and misunderstandings to be identified sooner. This improves communication between developers and testers and reduces the cost and effort of fixing defects later.

23. Test Coverage

A system that fails during peak usage because important scenarios were not tested indicates inadequate test coverage. Testers should identify different usage conditions and ensure that important functional, performance, boundary, and exceptional scenarios are included in the test suite. For a system expecting many simultaneous users, load and stress conditions should also be tested. Better test coverage increases the probability of finding failures before production. It also provides greater confidence that the system can operate correctly under expected and unexpected conditions.

24. Defect Documentation

Proper defect documentation is essential for effective defect management. Each defect should contain sufficient information such as its description, steps to reproduce, expected result, actual result, severity, priority, affected module, and current status. Maintaining this information in a defect repository allows testers and developers to track defects from identification to closure. It also helps identify recurring problems and supports historical analysis. Good documentation improves communication, prevents defects from being forgotten, and makes the overall defect resolution process more organized.

25. Structured Testing in Banking Systems

Repeated transaction failures in a banking system indicate that the existing testing approach is not sufficiently structured. The organization should establish a systematic process covering requirement analysis, test planning, test case design, functional testing, integration testing, regression testing, and defect tracking. Banking transactions should be tested using normal, invalid, boundary, and failure conditions. Historical defects should also be considered during regression testing. A structured approach improves coverage and helps identify transaction-related problems before they reach customers, thereby increasing system reliability.

26. Edge Case and Boundary Testing

Testing only normal inputs is insufficient because many defects occur at the boundaries and under unusual conditions. Testers should include minimum, maximum, just-inside, just-outside, and invalid values in their

test cases. For example, if a withdrawal limit is ₹50,000, values around the limit should be tested along with invalid and zero amounts. Boundary testing helps reveal errors in conditions and validation logic. Including edge cases increases test coverage and improves the ability of the system to handle unusual inputs correctly.

27. Process-Based Testing

Process-based testing can improve consistency by defining common testing activities, procedures, standards, and documentation for all modules. Each team should follow a similar process for test planning, test case preparation, execution, defect reporting, retesting, and regression testing. Standardization ensures that one module is not tested thoroughly while another receives very little testing. It also makes results easier to compare and track. A consistent process therefore improves overall quality and reduces uneven testing across different modules.

28. Input Validation

A web application that fails when unexpected inputs are provided indicates inadequate input validation testing. Testers should verify how the application handles empty values, incorrect formats, special characters, excessive values, invalid data types, and unexpected inputs. For example, a numeric field should reject inappropriate text input and handle boundary values correctly. Testing these conditions helps identify validation weaknesses and prevents crashes or incorrect processing. Proper input validation improves the robustness, reliability, and security of the application.

29. Defect Leakage

Defect leakage occurs when defects that should have been identified during testing reach later stages or production. Frequent defect leakage may indicate inadequate test coverage, poor test case design, insufficient regression testing, unclear requirements, or weak communication. The organization should analyze leaked defects to identify their root causes and update test cases accordingly. Stronger requirement reviews, systematic testing, regression testing, and defect analysis can reduce leakage. This improves software quality and decreases the cost of fixing defects after release.

30. Early Defect Detection

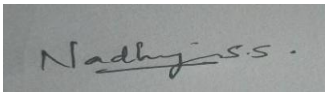
The testing strategy should focus on detecting defects as early as possible because the cost and effort of fixing defects generally increase in later stages. Testers should participate in requirement and design reviews and begin test planning early. Unit testing, integration testing, and continuous validation should then be

performed throughout development. For example, identifying an incorrect requirement during review requires much less effort than discovering the same issue after the complete system has been developed. Early defect detection reduces rework, saves time and cost, and improves the overall quality of the software.

Code of Conduct:

I Nadhiya SS (192321136) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	**30**	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	**25**	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	**20**	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	**15**	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	**10**	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand